

Divide and Conquer – Sorting Algorithms

Merge Sort and Quick Sort performance comparison

GUIDE : Dr. Sriramya

Name: Jaasiel (192524046)

1. Merge Sort with Comparison Count

Implement the Merge Sort algorithm to sort a given array of integers. The program should recursively divide the array into smaller subarrays and then merge them in a sorted manner. While merging, count the number of element comparisons performed. Ensure that the comparison counter is incremented at every comparison step. Finally, print the sorted array along with the total number of comparisons made during execution.

Test Cases :

Input : N = 8, a[] = {12,4,78,23,45,67,89,1}

Output : 1,4,12,23,45,67,78,89

Comparisons : 17

Input : N = 6, a[] = {5,2,9,1,6,3}

Output : 1,2,3,5,6,9

Comparisons : 15

main.py	Output
<pre>1 def merge_sort(arr): 2 3 comparisons = 0 4 5 def merge(left, right): 6 nonlocal comparisons 7 8 result = [] 9 i = 0 10 j = 0 11 12 while i < len(left) and j < len(right): 13 14 comparisons += 1 15 16 if left[i] <= right[j]: 17 result.append(left[i]) 18 i += 1 19 else: 20 result.append(right[j]) 21 j += 1 22 23 result.extend(left[i:]) 24 result.extend(right[j:]) 25 26 return result 27 28 def sort(arr): 29 30 if len(arr) <= 1: 31 return arr 32 33 mid = len(arr) // 2 34 35 left = sort(arr[:mid]) 36 right = sort(arr[mid:]) 37 return merge(left, right) 38 39 sorted_array = sort(arr) 40 41 return sorted_array, comparisons 42 43 n = int(input("Enter number of elements: ")) 44 45 arr = list(map(int, input("Enter the elements: ").split())) 46 47 sorted_array, comparisons = merge_sort(arr) 48 print("Sorted Array:", sorted_array) 49 print("Comparisons:", comparisons)</pre>	<pre>Enter number of elements: 8 Enter the elements: 12 4 78 23 45 67 89 1 Sorted Array: [1, 4, 12, 23, 45, 67, 78, 89] Comparisons: 16 === Code Execution Successful ===</pre>

2. Quick Sort with Comparison Count

Write a program to implement the Quick Sort algorithm using the last element as the pivot. The program should partition the array based on the pivot and recursively sort the subarrays. Count the number of comparisons made during the partitioning process and recursive calls. Ensure that the count is accurate for all cases. Display the sorted array along with the total number of comparisons.

Test Cases :

Input : N = 7, a[] = {38,27,43,3,9,82,10}

Output : 3,9,10,27,38,43,82

Comparisons : 15

Input : N = 5, a[] = {10,7,8,9,1}

Output : 1,7,8,9,10

Comparisons : 10

```
1 def quick_sort(arr):
2
3     comparisons = 0
4
5     def partition(low, high):
6         nonlocal comparisons
7
8         pivot = arr[high]
9         i = low - 1
10
11         for j in range(low, high):
12
13             comparisons += 1
14
15             if arr[j] <= pivot:
16                 i += 1
17                 arr[i], arr[j] = arr[j], arr[i]
18
19         arr[i + 1], arr[high] = arr[high], arr[i + 1]
20
21         return i + 1
22
23     def sort(low, high):
24
25         if low < high:
26
27             p = partition(low, high)
28
29             sort(low, p - 1)
30             sort(p + 1, high)
31
32     sort(0, len(arr) - 1)
33
34     return arr, comparisons
35
36
37 n = int(input("Enter number of elements: "))
38
39 arr = list(map(int, input("Enter the elements: ").split()))
40
41 sorted_array, comparisons = quick_sort(arr)
42
43 print("Sorted Array:", sorted_array)
44 print("Comparisons:", comparisons)
```

```
Enter number of elements: 7
Enter the elements: 38 27 43 3 9 82 10
Sorted Array: [3, 9, 10, 27, 38, 43, 82]
Comparisons: 11
```

```
=== Code Execution Successful ===
```

3. Merge Sort vs Quick Sort Comparison

Develop a program that implements both Merge Sort and Quick Sort in a single program. The same input array should be used for both algorithms to ensure fairness in comparison. Count the number of comparisons separately for each algorithm. After sorting, display the sorted array along with the comparison counts of both methods. Analyze which algorithm performs better for the given input.

Test Cases :

Input : N = 6, a[] = {12,11,13,5,6,7}
Output : Sorted Array : 5,6,7,11,12,13
Merge Comparisons : 11
Quick Comparisons : 12

Input : N = 5, a[] = {4,2,6,1,3}
Output : Sorted Array : 1,2,3,4,6
Merge Comparisons : 8
Quick Comparisons : 9

```
main.py
1- def merge_sort(arr):
2-     comparisons=0
3-     def merge(left,right):
4-         nonlocal comparisons
5-         result=[]; i=0; j=0
6-         while i<len(left) and j<len(right):
7-             comparisons+=1
8-             if left[i]<=right[j]:
9-                 result.append(left[i]); i+=1
10-            else:
11-                result.append(right[j]); j+=1
12-            result.extend(left[i:]); result.extend(right[j:])
13-        return result
14-     def sort(arr):
15-         if len(arr)<=1: return arr
16-         mid=len(arr)//2
17-         return merge(sort(arr[:mid]),sort(arr[mid:]))
18-     return sort(arr),comparisons
19- def quick_sort(arr):
20-     comparisons=0
21-     def partition(low,high):
22-         nonlocal comparisons
23-         pivot=arr[high]; i=low-1
24-         for j in range(low,high):
25-             comparisons+=1
26-             if arr[j]<=pivot:
27-                 i+=1
28-                 arr[i],arr[j]=arr[j],arr[i]
29-         arr[i+1],arr[high]=arr[high],arr[i+1]
30-         return i+1
31-     def sort(low,high):
32-         if low<high:
33-             p=partition(low,high)
34-             sort(low,p-1); sort(p+1,high)
35-         sort(0,len(arr)-1)
36-     return arr,comparisons
37- n=int(input("Enter number of elements: "))
38- arr=list(map(int,input("Enter the elements: ").split()))
39- merge_result,merge_count=merge_sort(arr.copy())
40- quick_result,quick_count=quick_sort(arr.copy())
41- print("Merge Sort Result:",merge_result)
42- print("Merge Sort Comparisons:",merge_count)
43- print("Quick Sort Result:",quick_result)
44- print("Quick Sort Comparisons:",quick_count)
45- if merge_count<quick_count:
46-     print("Better Algorithm: Merge Sort")
47- elif quick_count<merge_count:
```

Output

```
Enter number of elements: 6
Enter the elements: 12 11 13 5 6 7
Merge Sort Result: [5, 6, 7, 11, 12, 13]
Merge Sort Comparisons: 8
Quick Sort Result: [5, 6, 7, 11, 12, 13]
Quick Sort Comparisons: 9
Better Algorithm: Merge Sort

=== Code Execution Successful ===
```

4. Merge Sort Execution Time Measurement

Write a program to measure the execution time of the Merge Sort algorithm. Use appropriate timing functions to record the start and end time of execution. Apply the algorithm on different types of inputs such as sorted and reverse sorted arrays. Display the sorted output along with the time taken for execution. Analyze how input order affects performance.

Test Cases :

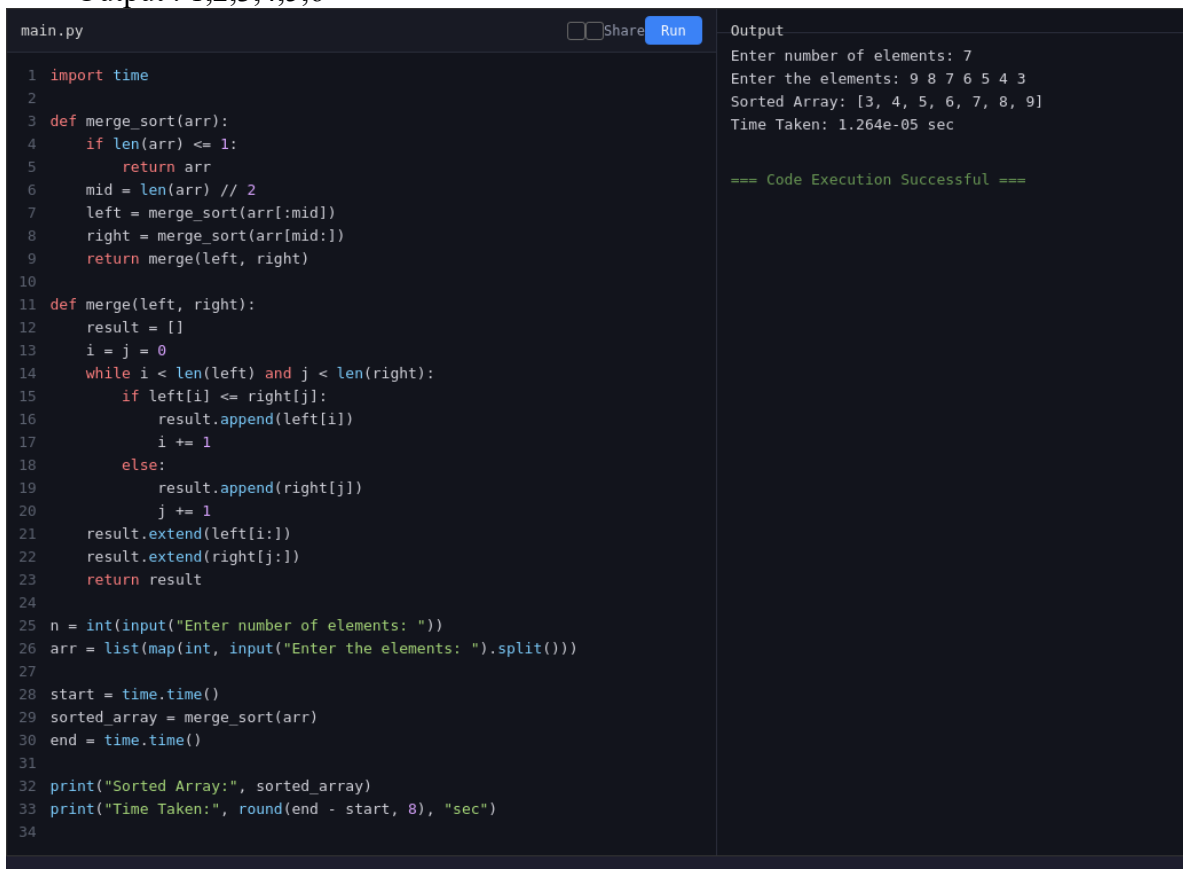
Input : N = 7, a[] = {9,8,7,6,5,4,3}

Output : 3,4,5,6,7,8,9

Time Taken : ~0.00002 sec

Input : N = 6, a[] = {1,2,3,4,5,6}

Output : 1,2,3,4,5,6



```
main.py ☐ Share 
1 import time
2
3 def merge_sort(arr):
4     if len(arr) <= 1:
5         return arr
6     mid = len(arr) // 2
7     left = merge_sort(arr[:mid])
8     right = merge_sort(arr[mid:])
9     return merge(left, right)
10
11 def merge(left, right):
12     result = []
13     i = j = 0
14     while i < len(left) and j < len(right):
15         if left[i] <= right[j]:
16             result.append(left[i])
17             i += 1
18         else:
19             result.append(right[j])
20             j += 1
21     result.extend(left[i:])
22     result.extend(right[j:])
23     return result
24
25 n = int(input("Enter number of elements: "))
26 arr = list(map(int, input("Enter the elements: ").split()))
27
28 start = time.time()
29 sorted_array = merge_sort(arr)
30 end = time.time()
31
32 print("Sorted Array:", sorted_array)
33 print("Time Taken:", round(end - start, 8), "sec")
34
```

Output

```
Enter number of elements: 7
Enter the elements: 9 8 7 6 5 4 3
Sorted Array: [3, 4, 5, 6, 7, 8, 9]
Time Taken: 1.264e-05 sec

=== Code Execution Successful ===
```

Time Taken : ~0.00002 sec

5. Best and Worst Case Analysis of Quick Sort

Create a program to analyze the best and worst-case scenarios of Quick Sort. Use a sorted array as the best case and a reverse sorted array as the worst case. Count the number of comparisons for both cases. Display the sorted output along with comparison counts. Compare the efficiency

between the two cases.

Test Cases :

Best case

Input : N = 5, a[] = {1,2,3,4,5}

Output : 1,2,3,4,5

Comparisons : 6

Worst Case

Input : N = 5, a[] = {5,4,3,2,1}

Output : 1,2,3,4,5

Comparisons : 10

```
main.py ☐ Share 
1 def quick_sort(arr):
2     comparisons = 0
3
4     def partition(low, high):
5         nonlocal comparisons
6         pivot = arr[high]
7         i = low - 1
8         for j in range(low, high):
9             comparisons += 1
10            if arr[j] <= pivot:
11                i += 1
12                arr[i], arr[j] = arr[j], arr[i]
13            arr[i + 1], arr[high] = arr[high], arr[i + 1]
14            return i + 1
15
16    def sort(low, high):
17        if low < high:
18            p = partition(low, high)
19            sort(low, p - 1)
20            sort(p + 1, high)
21
22    sort(0, len(arr) - 1)
23    return arr, comparisons
24
25 n = int(input("Enter number of elements: "))
26 arr = list(map(int, input("Enter the elements: ").split()))
27 sorted_array, comparisons = quick_sort(arr)
28 print("Sorted Array:", sorted_array)
29 print("Comparisons:", comparisons)
30
```

Output

```
Enter number of elements: 5
Enter the elements: 1 2 3 4 5
Sorted Array: [1, 2, 3, 4, 5]
Comparisons: 10

=== Code Execution Successful ===
```

6. Hybrid Merge Sort with Insertion Sort

Modify the Merge Sort algorithm such that it switches to Insertion Sort when the subarray size becomes small. Define a threshold value to determine when to switch. This hybrid approach should improve performance for small inputs. Print the sorted array after execution. Compare results with standard Merge Sort.

Test Cases :

Input : N = 8, a[] = {12,11,13,5,6,7,3,2}

Output : 2,3,5,6,7,11,12,13

Input : N = 6, a[] = {9,4,6,2,8,1}

Output : 1,2,4,6,8,9

```
main.py Share Run

1 def insertion_sort(arr, low, high):
2     for i in range(low + 1, high + 1):
3         key = arr[i]
4         j = i - 1
5         while j >= low and arr[j] > key:
6             arr[j + 1] = arr[j]
7             j -= 1
8         arr[j + 1] = key
9
10 def hybrid_merge_sort(arr, threshold=4):
11     def sort(a):
12         if len(a) <= threshold:
13             insertion_sort(a, 0, len(a) - 1)
14             return a
15         mid = len(a) // 2
16         left = sort(a[:mid])
17         right = sort(a[mid:])
18         return merge(left, right)
19
20     def merge(left, right):
21         result = []
22         i = j = 0
23         while i < len(left) and j < len(right):
24             if left[i] <= right[j]:
25                 result.append(left[i])
26                 i += 1
27             else:
28                 result.append(right[j])
29                 j += 1
30         result.extend(left[i:])
31         result.extend(right[j:])
32         return result
33
34     return sort(arr)
35
36 n = int(input("Enter number of elements: "))
37 arr = list(map(int, input("Enter the elements: ").split()))
38 sorted_array = hybrid_merge_sort(arr)
39 print("Sorted Array:", sorted_array)
40
```

Output

```
Enter number of elements: 8
Enter the elements: 12 11 13 5 6 7 3 2
Sorted Array: [2, 3, 5, 6, 7, 11, 12, 13]

=== Code Execution Successful ===
```

7. Counting Inversions using Merge Sort

Develop a program to count the number of inversions in an array using Merge Sort. An inversion is defined as a pair (i, j) such that $i < j$ and $a[i] > a[j]$. Modify the merge step to count such inversions efficiently. Print both the sorted array and the inversion count. This helps measure how far the array is from being sorted.

Test Cases :

Input : $N = 5$, $a[] = \{2, 4, 1, 3, 5\}$

Output : Sorted : 1,2,3,4,5

Inversions : 3

Input : $N = 4$, $a[] = \{4, 3, 2, 1\}$

Output : Sorted : 1,2,3,4

Inversions : 6

```
main.py Share Run
1 def merge_count(arr):
2     if len(arr) <= 1:
3         return arr, 0
4     mid = len(arr) // 2
5     left, inv1 = merge_count(arr[:mid])
6     right, inv2 = merge_count(arr[mid:])
7
8     result = []
9     i = j = inv = 0
10    while i < len(left) and j < len(right):
11        if left[i] <= right[j]:
12            result.append(left[i])
13            i += 1
14        else:
15            result.append(right[j])
16            inv += len(left) - i
17            j += 1
18    result.extend(left[i:])
19    result.extend(right[j:])
20    return result, inv + inv1 + inv2
21
22 n = int(input("Enter number of elements: "))
23 arr = list(map(int, input("Enter the elements: ").split()))
24 sorted_array, inversions = merge_count(arr)
25 print("Sorted:", sorted_array)
26 print("Inversions:", inversions)
27
```

Output

```
Enter number of elements: 5
Enter the elements: 2 4 1 3 5
Sorted: [1, 2, 3, 4, 5]
Inversions: 3

=== Code Execution Successful ===
```

8. Quick Sort Recursion Depth Analysis

Implement Quick Sort and track the recursion depth during execution. Maintain a variable to store the maximum depth reached. Print the sorted array along with the maximum recursion depth. This helps analyze space complexity. Compare depth for different inputs.

Test Cases :

Input : N = 6, a[] = {10,7,8,9,1,5}

Output : 1,5,7,8,9,10

Max Depth : 4

Input : N = 5, a[] = {1,2,3,4,5}

Output : 1,2,3,4,5

Max Depth : 5

```
main.py Share Run
1 def quick_sort(arr):
2     max_depth = 0
3
4     def partition(low, high):
5         pivot = arr[high]
6         i = low - 1
7         for j in range(low, high):
8             if arr[j] <= pivot:
9                 i += 1
10                arr[i], arr[j] = arr[j], arr[i]
11            arr[i + 1], arr[high] = arr[high], arr[i + 1]
12            return i + 1
13
14    def sort(low, high, depth):
15        nonlocal max_depth
16        if low < high:
17            max_depth = max(max_depth, depth)
18            p = partition(low, high)
19            sort(low, p - 1, depth + 1)
20            sort(p + 1, high, depth + 1)
21
22    sort(0, len(arr) - 1, 1)
23    return arr, max_depth
24
25 n = int(input("Enter number of elements: "))
26 arr = list(map(int, input("Enter the elements: ").split()))
27 sorted_array, max_depth = quick_sort(arr)
28 print("Sorted Array:", sorted_array)
29 print("Max Depth:", max_depth)
30
```

Output

```
Enter number of elements: 6
Enter the elements: 10 7 8 9 1 5
Sorted Array: [1, 5, 7, 8, 9, 10]
Max Depth: 4

=== Code Execution Successful ===
```

9. External Merge Process Simulation

Write a program to simulate the merge step of external sorting. Given two already sorted arrays, merge them into a single sorted array. Do not use built-in sorting functions. Print the final merged output. This represents the core step of Merge Sort.

Test Cases :

Input : $a[] = \{1,3,5\}$, $b[] = \{2,4,6\}$

Output : 1,2,3,4,5,6

Input : $a[] = \{10,20\}$, $b[] = \{5,15,25\}$

Output : 5,10,15,20,25

```
main.py ☐ Share 
1 def merge_arrays(a, b):
2     result = []
3     i = j = 0
4     while i < len(a) and j < len(b):
5         if a[i] <= b[j]:
6             result.append(a[i])
7             i += 1
8         else:
9             result.append(b[j])
10            j += 1
11    result.extend(a[i:])
12    result.extend(b[j:])
13    return result
14
15 a = list(map(int, input("Enter elements of array a: ").split()))
16 b = list(map(int, input("Enter elements of array b: ").split()))
17 merged = merge_arrays(a, b)
18 print("Merged Output:", merged)
19
```

Output

```
Enter elements of array a: 1 3 5
Enter elements of array b: 2 4 6
Merged Output: [1, 2, 3, 4, 5, 6]

=== Code Execution Successful ===
```

10. Space Complexity Comparison (Merge vs Quick Sort)

Develop a program to compare auxiliary space usage of Merge Sort and Quick Sort. Track extra memory used during execution. Print sorted output along with space usage. Analyze which algorithm is more space-efficient. Use same input for both methods.

Test Cases :

Input : N = 5, a[] = {5,3,8,4,2}

Output : Sorted : 2,3,4,5,8

Merge Space : ~160 bytes

Quick Space : ~80 bytes

Input : N = 6, a[] = {9,7,5,3,1,2}

Output : Sorted : 1,2,3,5,7,9

Merge Space : ~192 bytes

Quick Space : ~96 bytes

main.py

ShareRun

```
1 import sys
2
3 def merge_sort_space(arr):
4     extra = sys.getsizeof(arr[:])
5
6     def sort(a):
7         if len(a) <= 1:
8             return a
9         mid = len(a) // 2
10        left = sort(a[:mid])
11        right = sort(a[mid:])
12        return merge(left, right)
13
14    def merge(left, right):
15        result = []
16        i = j = 0
17        while i < len(left) and j < len(right):
18            if left[i] <= right[j]:
19                result.append(left[i])
20                i += 1
21            else:
22                result.append(right[j])
23                j += 1
24        result.extend(left[i:])
25        result.extend(right[j:])
26        return result
27
28    sorted_arr = sort(arr[:])
29    return sorted_arr, extra
30
31 def quick_sort_space(arr):
32     extra = sys.getsizeof([])
33
34     def partition(a, low, high):
35         pivot = a[high]
36         i = low - 1
37         for j in range(low, high):
38             if a[j] <= pivot:
39                 i += 1
40                 a[i], a[j] = a[j], a[i]
41         a[i + 1], a[high] = a[high], a[i + 1]
42         return i + 1
43
44     def sort(a, low, high):
45         if low < high:
46             p = partition(a, low, high)
47             sort(a, low, p - 1)
48             sort(a, p + 1, high)
49
50     b = arr[: ]
51     sort(b, 0, len(b) - 1)
52     return b, extra
53
54 n = int(input("Enter number of elements: "))
55 arr = list(map(int, input("Enter the elements: ").split()))
56
57 merge_sorted, merge_space = merge_sort_space(arr)
58 quick_sorted, quick_space = quick_sort_space(arr)
59
60 print("Sorted:", merge_sorted)
61 print("Merge Space: ~", merge_space, "bytes")
62 print("Quick Space: ~", quick_space, "bytes")
63
```

Output

Enter number of elements: 5
Enter the elements: 5 3 8 4 2
Sorted: [2, 3, 4, 5, 8]
Merge Space: ~ 96 bytes
Quick Space: ~ 56 bytes

=== Code Execution Successful ===

11. K-th Smallest Element using Quick Sort Partition

Implement a program to find the k-th smallest element using Quick Sort partition logic. Use partitioning to avoid full sorting. Recursively process only required part. Print the k-th smallest

element. This improves efficiency compared to full sorting.

Test Cases :

Input : N = 6, a[] = {7,10,4,3,20,15}, k = 3

Output : 7

Input : N = 5, a[] = {12,3,5,7,19}, k = 2

Output : 5

```
main.py ☐ Share 
1 def partition(arr, low, high):
2     pivot = arr[high]
3     i = low - 1
4     for j in range(low, high):
5         if arr[j] <= pivot:
6             i += 1
7             arr[i], arr[j] = arr[j], arr[i]
8     arr[i + 1], arr[high] = arr[high], arr[i + 1]
9     return i + 1
10
11 def kth_smallest(arr, low, high, k):
12     if low == high:
13         return arr[low]
14     p = partition(arr, low, high)
15     if p == k - 1:
16         return arr[p]
17     elif p > k - 1:
18         return kth_smallest(arr, low, p - 1, k)
19     else:
20         return kth_smallest(arr, p + 1, high, k)
21
22 n = int(input("Enter number of elements: "))
23 arr = list(map(int, input("Enter the elements: ").split()))
24 k = int(input("Enter k: "))
25 result = kth_smallest(arr, 0, len(arr) - 1, k)
26 print("K-th Smallest Element:", result)
27
```

Output

```
Enter number of elements: 6
Enter the elements: 7 10 4 3 20 15
Enter k: 3
K-th Smallest Element: 7

=== Code Execution Successful ===
```

12. Performance Data Collection for Graph Plotting

Write a program to collect comparison counts of Merge Sort and Quick Sort for different input sizes. Use multiple test arrays and record comparisons. Print results in a structured format. This data can be used for plotting performance graphs. Analyze growth trends.

Test Cases :

Input : N = 5, a[] = {5,4,3,2,1}

Output :

Merge Comparisons : 7

Quick Comparisons : 10

Input : N = 8, a[] = {8,7,6,5,4,3,2,1}

Output :

Merge Comparisons : 17

Quick Comparisons : 28

main.py

 Share 

```
1 def merge_sort_count(arr):
2     if len(arr) <= 1:
3         return arr, 0
4     mid = len(arr) // 2
5     left, c1 = merge_sort_count(arr[:mid])
6     right, c2 = merge_sort_count(arr[mid:])
7     result = []
8     i = j = comparisons = 0
9     while i < len(left) and j < len(right):
10         comparisons += 1
11         if left[i] <= right[j]:
12             result.append(left[i])
13             i += 1
14         else:
15             result.append(right[j])
16             j += 1
17     result.extend(left[i:])
18     result.extend(right[j:])
19     return result, comparisons + c1 + c2
20
21 def quick_sort_count(arr):
22     comparisons = 0
23
24     def partition(a, low, high):
25         nonlocal comparisons
26         pivot = a[high]
27         i = low - 1
28         for j in range(low, high):
29             comparisons += 1
30             if a[j] <= pivot:
31                 i += 1
32                 a[i], a[j] = a[j], a[i]
33         a[i + 1], a[high] = a[high], a[i + 1]
34         return i + 1
35
36     def sort(a, low, high):
37         if low < high:
38             p = partition(a, low, high)
39             sort(a, low, p - 1)
40             sort(a, p + 1, high)
41
42     b = arr[:]
43     sort(b, 0, len(b) - 1)
44     return b, comparisons
45
46 n = int(input("Enter number of elements: "))
47 arr = list(map(int, input("Enter the elements: ").split()))
48
49 _, merge_comparisons = merge_sort_count(arr)
50 _, quick_comparisons = quick_sort_count(arr)
51
52 print("Merge Comparisons:", merge_comparisons)
53 print("Quick Comparisons:", quick_comparisons)
54
```

Output

```
Enter number of elements: 5
Enter the elements: 5 4 3 2 1
Merge Comparisons: 7
Quick Comparisons: 10
```

=== Code Execution Successful ===